



Gráficos de líneas

Los gráficos de línea han sido el punto de partida para que comprendas las funciones básicas de trazado, la diversidad de sus parámetros y el manejo de la estética. Los que hemos construido hasta ahora, muestran la relación entre dos variables (año x indicador), considerando un *data-frame* de dos columnas con unos pocos registros. Pero la realidad es que la mayoría de tus *dataframes* serán más complejos que eso.

Si pensamos nuevamente en los indicadores de desarrollo mundial, todos ellos incluirán varios registros con la información de cada país, pero también varias columnas con el registro histórico a través de los años.

Retomemos el archivo **Population.csv** (ubicado en el apartado “Archivos adjuntos” de la plataforma) que hemos almacenado anteriormente en el *dataframe* `population`:



	Country Name	Country Code	1960	1961	1962	...	2019	2020	2021	2022	2023
0	Aruba	ABW	54922.0	55578.0	56320.0	...	109203	108587.0	107700	107310.0	107359
1	Africa Eastern and Southern	AFE	130072080.0	133534923.0	137171659.0	...	675950189	694446100.0	713090928	731821393.0	750503764
2	Afghanistan	AFG	9035043.0	9214083.0	9404406.0	...	37856121	39068979.0	40000412	40578842.0	41454761
3	Africa Western and Central	AFW	97630925.0	99706674.0	101854756.0	...	463365429	474569351.0	485920997	497387180.0	509398589
4	Angola	AGO	5231654.0	5301583.0	5354310.0	...	32375632	33451132.0	34532429	35635029.0	36749906
...	...	...	...	...	...	...	...	...	...	...	...
260	Kosovo	XKX	984846.0	1011421.0	1036950.0	...	1788891	1790152.0	1786080	1768096.0	1756366
261	Yemen, Rep.	YEM	5532301.0	5655232.0	5782221.0	...	35111408	36134863.0	37140230	38222876.0	39390799
262	South Africa	ZAF	16440172.0	16908035.0	17418522.0	...	59587885	60562381.0	61502603	62378410.0	63212384
263	Zambia	ZMB	3153729.0	3254086.0	3358099.0	...	18513839	19059395.0	19603607	20152938.0	20723965
264	Zimbabwe	ZWE	3809389.0	3930401.0	4055959.0	...	15271368	15526888.0	15797210	16069056.0	16340822

265 rows x 66 columns

El índice representa las posiciones consecutivas que emplea cada registro, en este caso, cada país: (0, 1, 2, . . . , 263); y las columnas, el nombre del país (`Country Name`), su código (`Country Code`) y los años en que hay información del número de habitantes (1960, 1961, . . . , 2023).

Ahora imagina que quieres construir un gráfico para comparar las poblaciones de los tres países de Norteamérica a lo largo del tiempo, ¿qué información de la almacenada en el *dataframe* es irrelevante?, ¿cómo convendría organizar el *dataframe*?

```
north_america = population[population['Country Name'].isin(['Mexico', 'United States', 'Canada'])]
```

	Country Name	Country Code	1960	1961	1962	...	2019	2020	2021	2022	2023
35	Canada	CAN	17909356.0	18271000.0	18614000.0	...	37618495	38028638.0	38239864	38939056.0	40097761
153	Mexico	MEX	36724615.0	37890118.0	39129200.0	...	125762982	126799054.0	127648148	128613117.0	129739759
250	United States	USA	180671000.0	183691000.0	186538000.0	...	328329953	331526933.0	332048977	333271411.0	334914895

3 rows × 66 columns

Queda claro que el índice ya no es tan representativo, pues ya ni siquiera indica la posición de cada registro en el *dataframe*. Entonces:

1. Hagamos que el nombre sea el nuevo índice:

```
north_america.set_index('Country Name', inplace=True)
```

2. Eliminemos el código del país, pues con el nombre será suficiente para identificarlo:

```
north_america = north_america.drop('Country Code', axis=1)
```

3. Si queremos que los años queden en el eje **X**, como hasta ahora, conviene aplicar la transpuesta y renombrar ambas dimensiones del *dataframe* resultante:

```
north_america = north_america.T
north_america.index.name = 'Year'
north_america.columns.name = ''
```



	Canada	Mexico	United States
Year			
1960	17909356.0	36724615.0	180671000.0
1961	18271000.0	37890118.0	183691000.0
1962	18614000.0	39129200.0	186538000.0
1963	18964000.0	40424065.0	189242000.0
1964	19325000.0	41772049.0	191889000.0
...	...	...	...
2019	37618495.0	125762982.0	328329953.0
2020	38028638.0	126799054.0	331526933.0
2021	38239864.0	127648148.0	332048977.0
2022	38939056.0	128613117.0	333271411.0
2023	40097761.0	129739759.0	334914895.0

64 rows × 3 columns

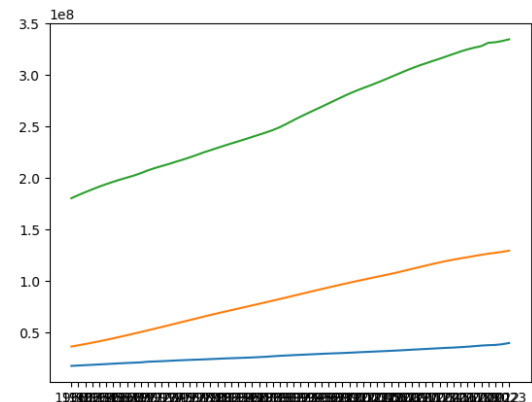
En la lección anterior utilizamos los métodos `plot()` y `lineplot()` para crear gráficos de línea, especificando manualmente qué variables colocar en cada eje. En todos los ejemplos, utilizamos el índice (los años) como eje *X* y los valores de población de México en el eje *Y*. Este enfoque funciona bien cuando se desea graficar una sola serie. Pero ¿qué ocurre si el *dataframe* contiene varias columnas numéricas y queremos visualizarlas todas en un mismo gráfico?

En el caso de pandas, ya habíamos visto que basta con llamar al método `plot()` sobre el *dataframe* para generar un gráfico, por lo que, si el *dataframe* contiene varias columnas numéricas, estas se graficarán automáticamente. Por otro lado, si consultas la documentación oficial de matplotlib y seaborn (localizada en el material adicional de este documento), observarás que ambas plataformas ofrecen un parámetro llamado `data` en sus funciones de trazado. Este parámetro permite trabajar de manera más conveniente con objetos que contienen datos etiquetados, como los *dataframes*, en lugar de pasarle los datos en *X* y *Y*.

Veamos la diferencia de lo que hicimos anteriormente para una columna en particular y el *dataframe* completo:

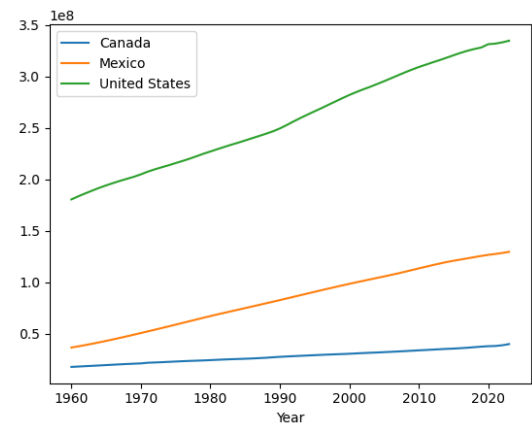
Con matplotlib

```
plt.plot(df.index, df[column])  
  
plt.plot(north_america)
```



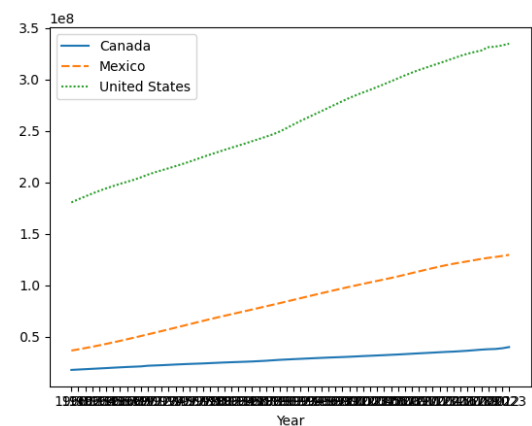
Con pandas

```
df.plot()  
  
north_america.plot()
```



Con seaborn

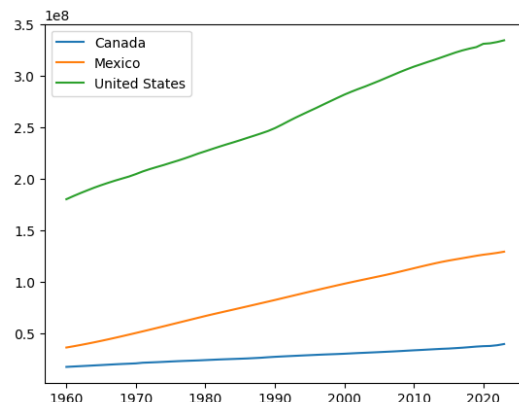
```
sns.lineplot(x=df.index, y=df[column])  
  
sns.lineplot(data=north_america)
```



De lo anterior, puedes identificar que matplotlib no pone por defecto la leyenda para saber qué serie corresponde a cada país. Además, tanto matplotlib como seaborn incluyen en el eje **X** las marcas de todos los años registrados, mientras que pandas lo hace en intervalos de 10 años. Para resolver esto, es necesario cambiar la frecuencia de las marcas en el eje **X** con la función `xticks()`.

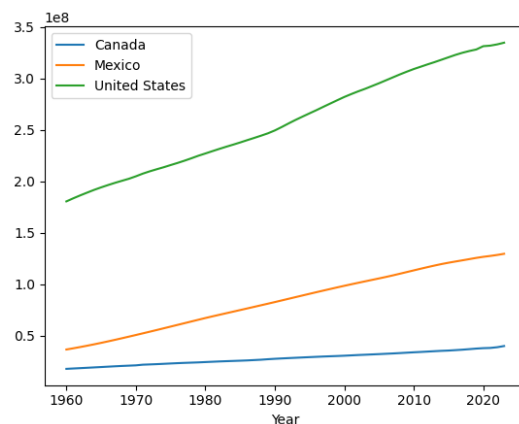
Con matplotlib

```
plt.plot(north_america)
plt.xticks(np.arange(0, len(north_america)+1, 10))
plt.legend(north_america.columns)
```



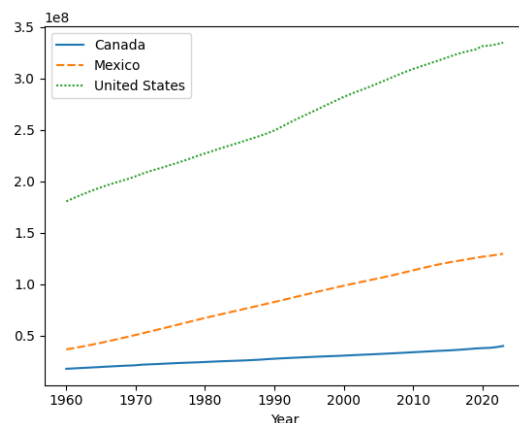
Con pandas

```
north_america.plot()
```



Con seaborn

```
sns.lineplot(data=north_america)
plt.xticks(np.arange(0, len(north_america)+1, 10))
```



La posibilidad de pasar el *dataframe* como argumento a las funciones de trazo también es válida cuando se desea graficar una única columna. A continuación, veremos cómo hacerlo utilizando las dos formas que hemos mencionado:

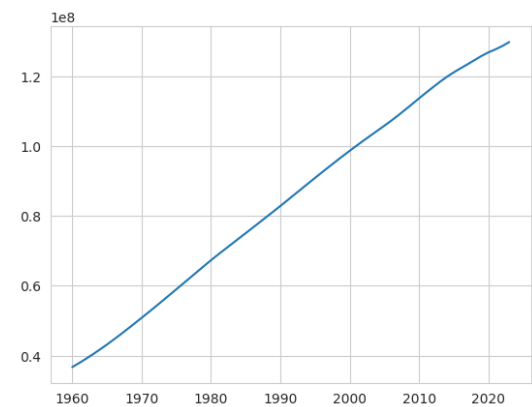
1. Especificando directamente las variables **X** y **Y** dentro de la función de graficación.
2. Utilizando el parámetro `data` para enviar el *dataframe* completo y referenciar las columnas por su nombre.

Con matplotlib

```
plt.plot(north_america.index,  
north_america['Mexico'])
```

o

```
plt.plot(north_america.  
index, 'Mexico', data=north_america)
```

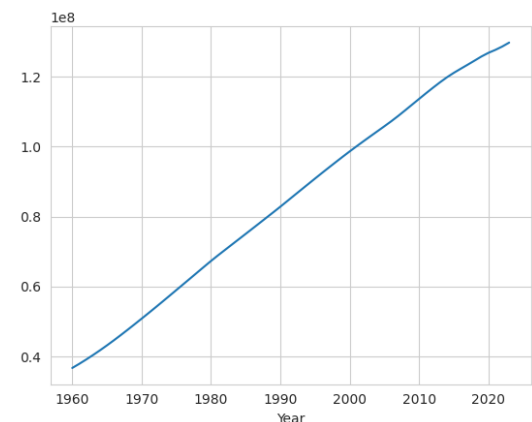


Se empleó el estilo *whitegrid*:

```
sns.set_style('whitegrid')
```

Con pandas

```
north_america['Mexico'].plot()  
north_america.plot(y='Mexico')
```

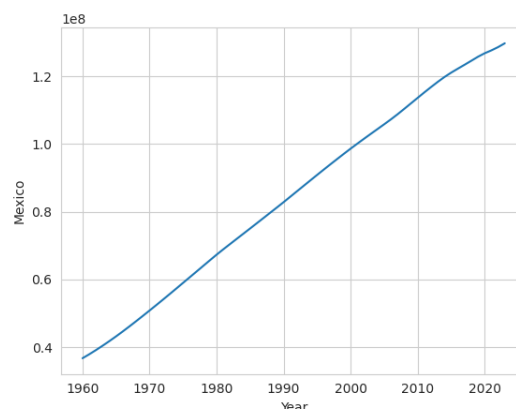


Con seaborn

```
sns.lineplot(x=north_america.index,  
y=north_america['Mexico'])
```

o

```
sns.lineplot(x=north_america.index,  
y='Mexico',data=north_america)
```



Nota: En los gráficos de matplotlib y seaborn es necesario realizar el ajuste de los años en el eje **X** utilizando `xticks`.

Si en el eje **X** no estuviera el índice, bastaría con especificar el nombre de la columna correspondiente, de la misma manera en que lo hacemos con el eje **Y**.

	Year	Canada	Mexico	United States
0	1960	17909009.0	37771859.0	180671000.0
1	1961	18271000.0	38966056.0	183691000.0
2	1962	18614000.0	40195319.0	186538000.0
3	1963	18964000.0	41462369.0	189242000.0
4	1964	19325000.0	42771079.0	191889000.0
...	...	...	...	...
55	2015	35702908.0	121858258.0	320635163.0
56	2016	36109487.0	123333376.0	322941311.0
57	2017	36543321.0	124777324.0	324985539.0
58	2018	37057765.0	126190788.0	326687501.0
59	2019	37589262.0	127575529.0	328239523.0

Como habrás observado en todos los códigos anteriores:

- La API `plot()` de pandas permite crear de forma rápida y sencilla cualquier gráfico a partir de un *dataframe*.
- La mayoría de los parámetros utilizados hasta ahora en la función `plot()` de matplotlib son posicionales, lo que significa que se interpretan en el orden en que se indican. Así, por ejemplo, para la llamada de abajo, por la posición se sabe que `north_america.index` irá en el eje **X** y `north_america['Mexico']` en el eje **Y**.

```
plt.plot(north_america.index, north_america['Mexico'])
```

- Todos los parámetros enviados a la función `lineplot()` de seaborn son con nombre, lo que significa que se pueden indicar en cualquier orden, por lo que hay que incluir el nombre de cada parámetro en la llamada. Por ejemplo, las dos llamadas siguientes producen el mismo gráfico:

```
sns.lineplot(x=north_america.index, y=north_america['Mexico'])
sns.lineplot(y=north_america['Mexico'], x=north_america.index)
```

En ambas se sabe que `north_america.index` irá en el eje **X** (porque se asigna a **X**) y `north_america['Mexico']` en el eje **Y**.

- Cuando una función combina atributos posicionales y por nombre, podrás diferenciarlos en la documentación oficial (localizada en el material adicional de este documento) con el símbolo de asignación (`=`). Por ejemplo, el encabezado de la función `plot()` de matplotlib está descrito como sigue:

```
plot([x], y, [fmt], *, data=None, **kwargs)
```

Lo que significa que `x,y,fmt` son atributos posicionales y `data` es por nombre. Los atributos por nombre siempre deben ir a la derecha de los posicionales. Los corchetes significan que el parámetro es opcional. Esto se refleja en la siguiente llamada:

```
plt.plot(north_america.index,'Mexico', data=north_america)
```

Por último, revisemos el efecto de las agrupaciones sobre los datos.

Cuando se utilizan *dataframes* de gran volumen, o se combinan varios de ellos, es común que la visualización se lleve a cabo sobre el resultado de agrupar registros. Por ello es muy común emplear la función `groupby()` previo al trazado, con alguna operación sobre los datos (`count`, `sum`, `mean`,...). La salida agregada de `groupby` produce como índices las etiquetas de grupo, lo que impacta también la graficación.

Para probar lo anterior, combinemos el *dataframe* `population` con el contenido del archivo **Metadata.csv** (localizado en el apartado “Archivos adjuntos” de la plataforma), leído en un *dataframe* con el mismo nombre (`metadata`). Este archivo contiene los países del indicador de población, categorizados por la región y el grupo de ingresos al que pertenecen.



	Country	Code	Region	IncomeGroup
0	ABW	Latin America & Caribbean		High income
1	AFE		NaN	Low income
2	AFG		South Asia	Low income
3	AFW		NaN	Low income
4	AGO		Sub-Saharan Africa	Lower middle income
...	...		...	...
260	XKX		Europe & Central Asia	Upper middle income
261	YEM		Middle East & North Africa	Low income
262	ZAF		Sub-Saharan Africa	Upper middle income
263	ZMB		Sub-Saharan Africa	Lower middle income
264	ZWE		Sub-Saharan Africa	Lower middle income

265 rows x 3 columns

Como puedes observar, ambos *dataframes* poseen la columna `Country Code`, así que el `merge()` lo podemos hacer por dicha columna:

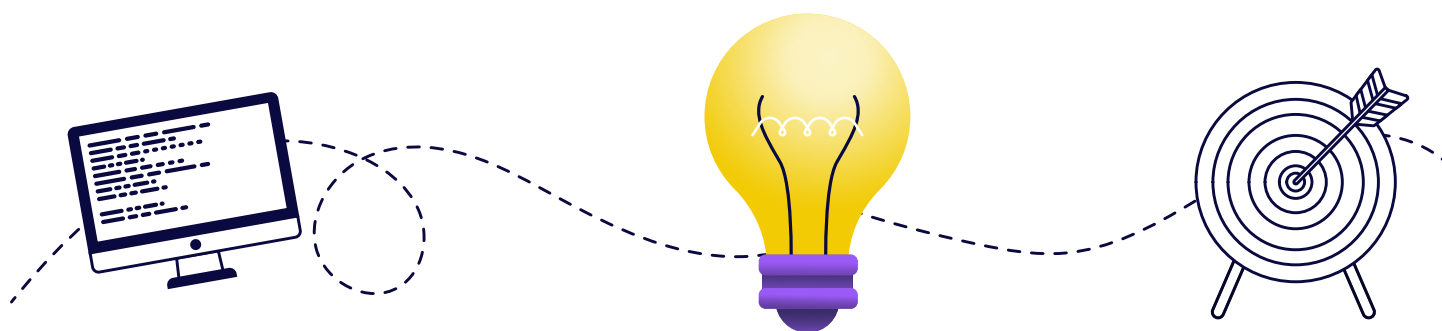
```
merged_df = population.merge(metadata, on='Country Code')
```

El *dataframe* obtenido queda como se muestra a continuación:



	Country Name	Country Code	1960	1961	1962	...	2021	2022	2023	Region	IncomeGroup
0	Aruba	ABW	54922.0	55578.0	56320.0	...	107700	107310.0	107359	Latin America & Caribbean	High income
1	Africa Eastern and Southern	AFE	130072080.0	133534923.0	137171659.0	...	713090928	731821393.0	750503764	NaN	Low income
2	Afghanistan	AFG	9035043.0	9214083.0	9404406.0	...	40000412	40578842.0	41454761	South Asia	Low income
3	Africa Western and Central	AFW	97630925.0	99706674.0	101854756.0	...	485920997	497387180.0	509398589	NaN	Low income
4	Angola	AGO	5231654.0	5301583.0	5354310.0	...	34532429	35635029.0	36749906	Sub-Saharan Africa	Lower middle income
...	...	...	...	...	...	...	...	...	...	...	...
260	Kosovo	XKX	984846.0	1011421.0	1036950.0	...	1786080	1768096.0	1756366	Europe & Central Asia	Upper middle income
261	Yemen, Rep.	YEM	5532301.0	5655232.0	5782221.0	...	37140230	38222876.0	39390799	Middle East & North Africa	Low income
262	South Africa	ZAF	16440172.0	16908035.0	17418522.0	...	61502603	62378410.0	63212384	Sub-Saharan Africa	Upper middle income
263	Zambia	ZMB	3153729.0	3254086.0	3358099.0	...	19603607	20152938.0	20723965	Sub-Saharan Africa	Lower middle income
264	Zimbabwe	ZWE	3809389.0	3930401.0	4055959.0	...	15797210	16069056.0	16340822	Sub-Saharan Africa	Lower middle income

265 rows × 68 columns



Nuestro objetivo será obtener el total de población por región para el año 2023. Utilicemos entonces `groupby()` de pandas con la función `sum()` para combinar todos los registros por grupo:

```
region_population = merged_df[['Region', '2023']].  
groupby('Region').sum()
```



2023

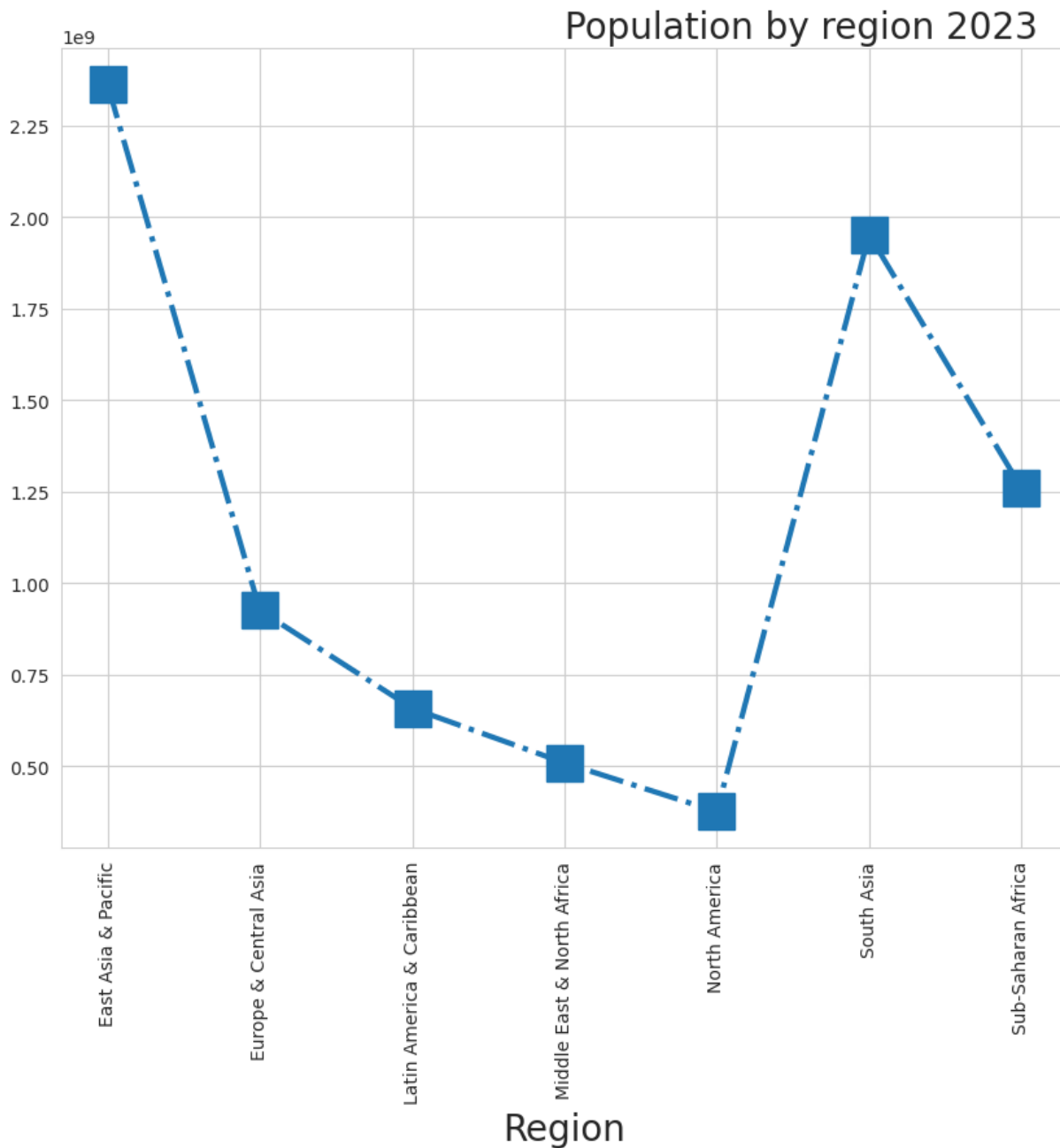
Region	
East Asia & Pacific	2361090328
Europe & Central Asia	924969865
Latin America & Caribbean	657611624
Middle East & North Africa	508311359
North America	375077354
South Asia	1951539835
Sub-Saharan Africa	1259902353

Si observas el resultado obtenido, el índice corresponde a la columna por la cual se realiza la agrupación, `Region` en este caso.

Con ello en mente, ya podrías graficar el *dataframe* resultante como lo has hecho antes. Por ejemplo, utilizando `pandas` y personalizando a través de `**kwargs` y las funciones de la interfaz `pyplot` de Matplotlib.

```
region_population.plot(rot=90,figsize=(10,8),legend=False,  
linestyle='-.',linewidth=3,marker='s',markersize=20)
```

```
plt.title('Population by region  
2019',fontsize=20,horizontalalignment='left')  
plt.xlabel('Region',fontsize=20)
```



Sin embargo, los gráficos de líneas están diseñados para mostrar tendencias a lo largo del tiempo. Cuando se trata de comparar valores agregados entre regiones en un momento específico, los gráficos de barras resultan más adecuados, ya que permiten visualizar con mayor claridad las diferencias entre categorías.

Antes de terminar...

Es muy probable que, en tu quehacer diario, la estructura u organización de tus fuentes de información sea muy diversa. Ahora que eres consciente del impacto que tienen los índices y los parámetros en las funciones de trazado, reflexiona lo siguiente:

- ¿Cómo podrías reestructurar tus *dataframes* para facilitar el proceso de graficación?
- ¿Cuándo es conveniente cambiar los índices o intercambiar filas y columnas?
- ¿Vas a graficar una o varias columnas?, ¿qué parámetros debes pasar en cada caso?
- ¿Qué información necesitas agrupar?, ¿por qué categoría?, ¿con qué medida de resumen?

Durante la siguiente semana, revisa la documentación oficial de las funciones `plot` en las tres plataformas, pero ahora haciendo énfasis en los parámetros que reciben. Esto te permitirá identificar sus tipos de datos, si son o no posicionales, si son opcionales o si tienen un valor por defecto.

Material adicional

● Matplotlib

The Matplotlib development team. (2025). *Matplotlib: Visualization with Python* [Matplotlib: Visualización con Python]. Matplotlib. Recuperado el 25 de abril de 2025 de: <https://matplotlib.org>

● Pandas

Pandas via. (2025). *Pandas*. Recuperado el 25 de abril de 2025 de: <https://pandas.pydata.org>

● Seaborn

Waskom, M. (2024). *Seaborn: statistical data visualization* [Seaborn: visualización de datos estadísticos]. Seaborn. Recuperado el 25 de abril de 2025 de: <https://seaborn.pydata.org>

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.

Glosario

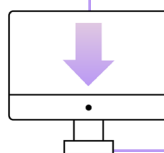
- **Gráficos de líneas:** Se utilizan para mostrar datos continuos o secuenciales, con líneas conectando puntos de datos a lo largo de un eje temporal o numérico.

Bibliografía

Melo, M. (26 de enero de 2024). *2023: aumento de llegadas de migrantes a México en ruta hacia EE. UU.* Statista. Recuperado el 25 de abril de 2025 de: <https://es.statista.com/grafico/16470/migrantes-registrados-y-devueltos-en-mexico/>

The World Bank Group. (2025). *Indicators* [Indicadores]. World Bank Group. Recuperado el 25 de abril de 2025 de: <https://data.worldbank.org/indicator>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.